

L'Agent IA

Architecture, État et Principes de Production



| Qu'est-ce qu'un agent IA ?

Ce n'est PAS

- ✗ Un prompt long complexe
- ✗ Un simple chatbot
- ✗ Un LLM "autonome" seul

C'est un PROGRAMME

Un agent est un système logiciel qui boucle sur lui-même, où le **LLM n'est qu'un composant** au service d'une logique métier.

| Définition Opérationnelle

- 🎯 **Poursuit un objectif explicite** : Le système est orienté vers un résultat final défini.
 - 💾 **Maintient un état** : Il possède une mémoire persistante de ses actions passées.
 - 🧠 **Décide de la prochaine action** : Il utilise le raisonnement pour planifier.
 - 🔧 **Agit via des outils** : Il interagit avec le monde réel (APIs, bases de données).
 - 👁️ **Observe le résultat** : Il analyse l'impact de ses actions.
 - 🔄 **S'auto-corrige** : Il itère jusqu'à atteindre un critère d'arrêt.
-

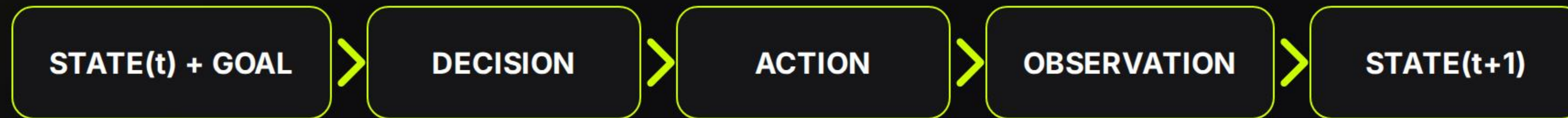
Architecture : Les 6 briques

Brique	Fonction
1. Objective	Mission à accomplir
2. State	Mémoire à l'instant t
3. Planner	Cerveau décisionnel (LLM)
4. Action Executor	Exécuteur technique (Outils)
5. Observer	Analyse des résultats
6. Controller	Condition d'arrêt

⚠ Supprimer une seule de ces briques → ce n'est plus un agent, juste une chaîne de prompts.

Modélisation Mentale

L'agent comme une **machine à états**.



Inspiré des systèmes de contrôle et des moteurs de workflow industriels.

| Le STATE : La Structure Centrale

Tout agent sérieux repose sur un state explicite.

Le LLM est amnésique ; le state est sa conscience persistante.

Exemple de state minimal (conceptuel) :

```
state = { "objective": "...", "plan": [], "memory": [], "errors": [], "done": False }
```



Le Planner : Le Cerveau

Seule partie où le LLM raisonne pour produire une **sortie structurée**.



Rôles du Planner

1. Analyser l'objectif.
2. Proposer l'action suivante.
3. Zéro texte libre.

```
# Sortie attendue (JSON) { "action": "search_web", "parameters": { "query": "EU AI regulations"
```


L'Action Executor

La séparation critique : L'agent propose, un composant exécute.

🛡️ Sécurité

🧪 Testabilité

🔧 Observabilité

```
def execute_action(action, params): if action == "search_web": retu
```



| L'Observer : L'Analyse

Ce que tout le monde oublie : l'interprétation de l'échec ou du succès pour mettre à jour le state.

- 🕒 Timeout API
- 👻 Résultat vide
- ⚠️ Contradictions

```
def observe(result, state): if result is None: state["errors"].append("Empty") else: state["memory"].append(result)
```



| Le Controller

Arrêter ou continuer. Un agent sans condition d'arrêt est dangereux.

STOP?

```
def should_stop(state): if len(state["errors"]) > 3: return True if objective_satisfied(state): return True return False
```

Critères d



La Boucle Principale

Le cœur de l'agent, sans magie.

```
while not state["done"]: decision = planner(state) result = execute(dec
```

Ceci est un agent. Le reste n'est qu'enrobage.

| Erreurs Classiques

- ✗ Laisser le LLM appeler directement des APIs
- ✗ Mélanger raisonnement et action
- ✗ Ne pas stocker l'état explicitement
- ✗ Absence de limite d'itérations
- ✗ Sorties non structurées

Merci de votre attention !

Image Sources



https://images.stockcake.com/public/f/e/8/fe827af7-0294-46ad-bab6-cab0542f0bd9_large/digital-data-hub-stockcake.jpg

Source: stockcake.com



https://images.stockcake.com/public/6/1/a/61a94265-8790-437c-8235-70f42e9d04a5_large/neon-robot-dance-stockcake.jpg

Source: stockcake.com



https://static.vecteezy.com/system/resources/thumbnails/071/755/800/small_2x/human-eye-scanning-with-advanced-digital-technology-in-the-near-future-video.jpg

Source: www.vecteezy.com
